

Predicción de Pit Stop en Fórmula 1 mediante Machine Learning

JERÓNIMO GÁLVEZ¹, CARLOS ALBERTO CALVACHE¹, JUAN DAVID CASTAÑEDA¹, AND SUSANA TORO CASTAÑO¹

¹Minería de Datos, Universidad Pontificia Bolivariana, 2026

Este proyecto desarrolla un proceso de minería de datos aplicado a la predicción de pit stop en Fórmula 1. El objetivo principal consiste en construir un modelo predictivo de clasificación capaz de anticipar si un piloto realizará un pit stop en la siguiente vuelta, usando variables históricas de carrera como compuesto de neumático, vida del neumático, posición, tiempo de vuelta, degradación acumulada y progreso de carrera. El conjunto de datos utilizado contiene 101.371 registros y 16 variables. Durante la preparación de datos se realizaron procesos de análisis exploratorio, limpieza, tratamiento de valores nulos, corrección de atípicos, análisis de correlaciones, selección de variables, balanceo de clases e ingeniería de características. Posteriormente, se implementaron modelos supervisados y de ensamble, evaluados mediante accuracy, precision, recall, F1-score y ROC-AUC. La selección de modelos se apoyó en ANOVA, prueba post-hoc de Tukey, GridSearchCV y optimización bayesiana. Finalmente, el mejor modelo fue almacenado en un pipeline y desplegado mediante una interfaz gráfica desarrollada en Streamlit.

1. INTRODUCCIÓN

La Fórmula 1 es un entorno altamente competitivo en el que las decisiones estratégicas pueden modificar el resultado de una carrera. Una de las decisiones más importantes es determinar el momento adecuado para realizar un *pit stop*, ya que esta acción afecta directamente el rendimiento del piloto, la vida útil de los neumáticos, la posición en pista y la estrategia general del equipo.

En este proyecto se estructura un proceso completo de minería de datos orientado a la predicción de pit stops. El problema se aborda como una tarea de clasificación binaria, donde la variable objetivo indica si un piloto realizará un pit stop en la siguiente vuelta.

El desarrollo fue realizado en Python mediante notebooks de preparación de datos, modelamiento, evaluación y despliegue.

Además, el modelo final fue integrado en un pipeline y usado en una aplicación de Streamlit, permitiendo convertir el análisis en una herramienta funcional para realizar predicciones.

Palabras clave: Fórmula 1, pit stop, clasificación, aprendizaje supervisado, minería de datos, Random Forest, Streamlit.

2. ENTENDIMIENTO DEL NEGOCIO

A. Descripción del negocio

La Fórmula 1 es la categoría máxima del automovilismo mundial. Cada temporada está compuesta por múltiples Grandes Premios en distintos circuitos. En este contexto, la estrategia de carrera es fundamental para lograr buenos resultados, especialmente en decisiones relacionadas con el momento de entrada a pits, el cambio de neumáticos y la administración del rendimiento del vehículo.

La decisión de realizar un pit stop no depende únicamente del desgaste del neumático. También intervienen factores como la posición del piloto, el compuesto utilizado, el avance de la carrera, el tiempo de vuelta, la degradación acumulada, los cambios de posición y las condiciones estratégicas de cada equipo. Por esta razón, el análisis de datos históricos puede aportar valor al permitir identificar patrones y apoyar la toma de decisiones.

B. Descripción del problema

Actualmente, la decisión sobre el momento de realizar un pit stop depende de la experiencia del ingeniero de carrera, de la telemetría y de los modelos internos de cada equipo. Sin embargo, anticipar si un piloto entrará a pits en la siguiente vuelta es un problema complejo debido a la interacción de múltiples variables de carrera.

El problema abordado consiste en predecir si un piloto realizará un pit stop en la siguiente vuelta a partir de datos históricos. Esta predicción puede ser útil para analizar estrategias, comprender patrones de comportamiento y construir una herramienta de apoyo basada en datos.

C. Objetivo general

Desarrollar un modelo predictivo de minería de datos en Python que permita estimar si un piloto de Fórmula 1 realizará un pit stop en la siguiente vuelta y desplegar el modelo final mediante una interfaz gráfica en Streamlit.

D. Objetivos específicos

- Realizar la preparación y limpieza del conjunto de datos, incluyendo tratamiento de nulos, atípicos y transformación de variables.
- Analizar la distribución de la variable objetivo y aplicar balanceo únicamente sobre el conjunto de entrenamiento.
- Entrenar y evaluar modelos supervisados y modelos de ensemble.
- Calcular métricas de evaluación como accuracy, precision, recall, F1-score y ROC-AUC.
- Seleccionar los mejores modelos mediante ANOVA y prueba post-hoc de Tukey.
- Optimizar los mejores modelos mediante GridSearchCV y optimización bayesiana.
- Construir un pipeline final con las operaciones de preparación y el modelo seleccionado.
- Desplegar el modelo final en una interfaz gráfica desarrollada con Streamlit.

E. Diseño de solución

Table 1. Diseño de solución del proyecto

Aspecto	Descripción
Problema	Predecir si un piloto realizará un pit stop en la siguiente vuelta.
Tipo de minería	Predictiva.
Tipo de análisis	Clasificación binaria.
Tipo de aprendizaje	Supervisado.
Datos requeridos	Histórico de vueltas, neumáticos, tiempos, degradación, posición y variable objetivo PitNextLap.
Métodos	Árbol de Decisión, KNN, Red Neuronal, SVM, Random Forest, Gradient Boosting, Bagging y AdaBoost.
Evaluación	Accuracy, precision, recall, F1-score, ROC-AUC, ANOVA y Tukey.

F. Recursos para creación del modelo y despliegue

Software:

- Python 3.10 o superior.
- Librerías: pandas, numpy, scikit-learn, scipy, imbalanced-learn, matplotlib, seaborn, pickle, streamlit y huggingface-hub.
- Entorno de desarrollo: Jupyter Notebook o Google Colab.
- Control de versiones: GitHub.
- Despliegue: Streamlit.

3. ENTENDIMIENTO DE LOS DATOS

A. Ciclo de los datos

Los datos corresponden a registros históricos vuelta a vuelta de carreras de Fórmula 1. Cada fila representa una vuelta realizada por un piloto, junto con información del neumático, posición, tiempo de vuelta, degradación y estado estratégico.

El dataset final usado en el proyecto contiene 101.371 registros y 16 variables. Esta cantidad de datos cumple con los requisitos

del proyecto, ya que supera los 400 registros mínimos y contiene más de 10 variables.

La variable objetivo es PitNextLap, la cual indica si el piloto realizará un pit stop en la siguiente vuelta. Esta variable transforma el problema en una tarea de clasificación binaria.

B. Diccionario de datos

Table 2. Diccionario de datos del proyecto

Variable	Descripción	Tipo
Driver	Abreviatura del piloto.	Catórica
LapNumber	Número de vuelta en la carrera.	Entera
Compound	Tipo de compuesto de neumático.	Catórica
Stint	Número de stint del piloto.	Entera
TyreLife	Vueltas de uso del neumático actual.	Continua
Position	Posición del piloto en carrera.	Entera
LapTime (s)	Tiempo de vuelta en segundos.	Continua
Race	Nombre del Gran Premio.	Catórica
Year	Año de la temporada.	Entera
LapTime_Delta	Diferencia del tiempo de vuelta frente a la vuelta anterior.	Continua
Cumulative_Degradation	Degradación acumulada desde el inicio del stint.	Continua
PitStop	Indica si hubo pit stop en la vuelta actual.	Binaria
PitNextLap	Indica si habrá pit stop en la siguiente vuelta.	Binaria
RaceProgress	Progreso de la carrera.	Continua
Normalized_TyreLife	Vida del neumático normalizada.	Continua
Position_Change	Cambio de posición frente a la vuelta anterior.	Continua

C. Reglas de calidad desde el negocio

Table 3. Reglas de calidad definidas desde el negocio

Variable	Regla de calidad
Driver	Debe corresponder a un piloto registrado.
LapNumber	Debe ser positivo y estar dentro del rango de vueltas.
Compound	Debe tomar valores SOFT, MEDIUM, HARD, INTERMEDIATE o WET.
Stint	Debe ser un entero positivo.
TyreLife	Debe ser mayor o igual a 1.
Position	Debe estar entre 1 y 20.
LapTime (s)	Debe ser un tiempo razonable; valores mayores a 150 segundos se revisan.
PitStop	Solo debe tomar valores 0 o 1.
PitNextLap	Solo debe tomar valores 0 o 1.
RaceProgress	Debe estar entre 0.0 y 1.0.
Normalized_TyreLife	Debe representar la vida del neumático en escala normalizada.

D. Pandas Profiling

Como parte del análisis exploratorio se generó un reporte automático de pandas profiling. Este reporte permitió obtener una visión general del conjunto de datos, incluyendo número de registros, tipos de variables, valores nulos, distribución de variables numéricas y catóricas, posibles alertas de calidad, correlaciones y comportamiento general de las variables.

El reporte fue útil para identificar la estructura del dataset, la presencia de valores faltantes en la variable Compound, la distribución de la variable objetivo PitNextLap y posibles valores atípicos en variables como LapTime (s) y Cumulative_Degradation.

4. ENLACES DEL PROYECTO

Repositorio en GitHub: [GitHub - Despliegue final minería de datos](#)

Aplicación desplegada en Streamlit: [Aplicación Streamlit - Predicción de Pit Stop](#)

5. PREPARACIÓN DE DATOS

A. Integración

El conjunto de datos se trabajó como una fuente consolidada. No fue necesario realizar uniones con otras tablas, debido a que el archivo final ya contenía las variables de carrera, neumáticos, posición, tiempos, degradación y variables derivadas.

B. Inspección inicial

Inicialmente se cargó el dataset en Python usando pandas. Se revisaron las primeras filas, los tipos de datos, la cantidad de registros y la cantidad de variables. El dataset contiene 101.371 registros y 16 variables. De estas variables, 7 son de tipo float64, 6 son de tipo int64 y 3 son de tipo object.

Table 4. Estructura general del conjunto de datos

Característica	Resultado
Número de registros	101.371
Número de variables	16
Variables numéricas	13
Variables categóricas	3
Variable objetivo	PitNextLap

C. Análisis de la variable objetivo

La variable objetivo del proyecto es PitNextLap. Esta variable es binaria, donde el valor 0 indica que el piloto no realizará un pit stop en la siguiente vuelta, mientras que el valor 1 indica que sí lo realizará.

Durante el análisis exploratorio se identificó un desbalanceo entre las clases. Este comportamiento es coherente con el contexto del problema, ya que en una carrera de Fórmula 1 la mayoría de vueltas no terminan en una entrada a pits. Por esta razón, en las etapas posteriores no se dependió únicamente del accuracy como métrica de evaluación. Se priorizaron métricas más adecuadas para problemas desbalanceados, como recall, precision, F1-score y ROC-AUC.

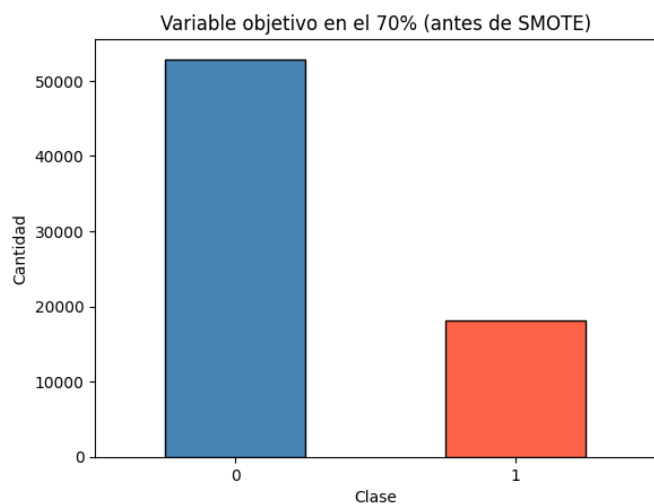


Fig. 1. Distribución de la variable objetivo PitNextLap.

D. Selección de variables

Para el modelo predictivo se seleccionaron como variables de entrada: Compound, Stint, TyreLife, Position, LapTime (s), LapTime_Delta, Cumulative_Degradation, RaceProgress, Normalized_TyreLife y Position_Change. La variable objetivo fue PitNextLap.

Se excluyeron variables como Driver, Race, Year, LapNumber y PitStop. Driver y Race pueden introducir alta cardinalidad; Year no aporta variabilidad suficiente; LapNumber está representada de forma más general por RaceProgress; y PitStop corresponde al evento de la vuelta actual, por lo que debe manejarse con cuidado para evitar fuga de información.

E. Revisión de valores nulos

Se revisaron los valores faltantes por variable. La mayoría de columnas no presentó datos nulos. La variable Compound presentó 66 valores nulos, una cantidad baja frente al total de registros. Dado que esta variable es relevante para la estrategia de neumáticos, fue tratada antes del modelamiento para evitar errores en la codificación.

Table 5. Resultado de revisión de valores nulos

Variable	Nulos	Acción
Driver	0	Sin tratamiento
LapNumber	0	Sin tratamiento
Compound	66	Tratamiento previo
Stint	0	Sin tratamiento
TyreLife	0	Sin tratamiento
Position	0	Sin tratamiento
LapTime (s)	0	Sin tratamiento
Race	0	Sin tratamiento
PitNextLap	0	Sin tratamiento

F. Limpieza de atípicos

Se identificaron 112 registros con LapTime (s) mayor a 150 segundos. Estos valores pueden estar asociados a safety car, entrada o salida de pits, tráfico, lluvia, errores o incidentes. Para reducir el impacto de estos registros extremos, se corrigieron usando la mediana del tiempo de vuelta por carrera.

Esta decisión permitió conservar los registros, evitando eliminar información que podría representar condiciones reales de carrera, pero reduciendo el efecto de valores excesivamente altos sobre los modelos.

G. Análisis de correlaciones

Se aplicó una matriz de correlación de Pearson entre variables numéricas. Se identificó una relación alta entre TyreLife y Normalized_TyreLife, lo cual es esperado porque la segunda variable es una transformación de la primera. También se identificó una relación aproximada de 0.65 entre Stint y RaceProgress, lo cual evidencia una redundancia moderada-alta entre ambas variables. Esta relación es coherente con el contexto del problema, ya que a medida que avanza la carrera también suele aumentar el número de stint en el que se encuentra el piloto. Además, se observó que ambas variables aportan información muy similar frente a la variable objetivo PitNextLap, debido a que las dos ayudan a representar el momento estratégico de la carrera en el que podría ocurrir una entrada a pits.

A pesar de esta redundancia, se decidió conservar ambas variables dentro del conjunto de entrenamiento. La razón principal es que, aunque *Stint* y *RaceProgress* están relacionadas y aportan información parecida a la variable objetivo, no representan exactamente el mismo concepto. *RaceProgress* describe el avance general de la carrera en una escala continua, mientras que *Stint* representa el segmento estratégico asociado al uso de neumáticos. Por lo tanto, mantener ambas variables permite que el modelo capture tanto el momento global de la carrera como la etapa específica de la estrategia de neumáticos. Esta decisión se consideró adecuada porque los modelos evaluados, especialmente los basados en árboles como *RandomForestClassifier*, pueden manejar variables correlacionadas sin afectar de forma crítica el desempeño predictivo.

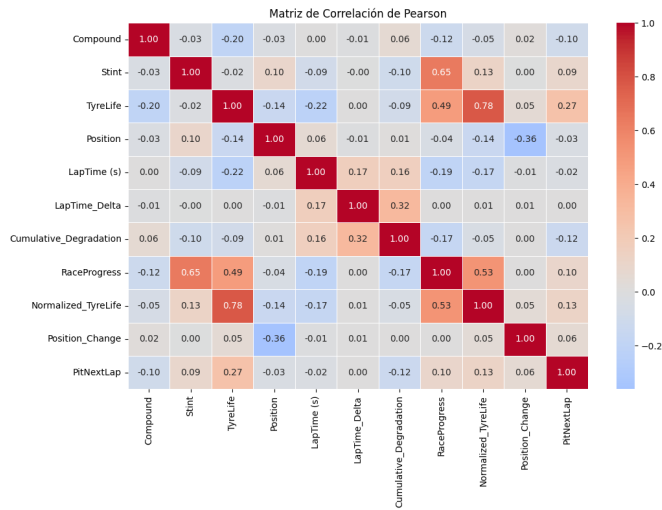


Fig. 2. Matriz de correlación entre variables numéricas.

Además, se aplicó correlación punto-biserial entre variables numéricas y la variable objetivo *PitNextLap*. La variable con mayor relación fue *TyreLife*, con una correlación de 0.2662 y p-valor igual a 0.00. Esto es coherente con el problema, porque a mayor vida del neumático, mayor probabilidad de que el piloto entre a pits próximamente.

Table 6. Correlación punto-biserial con *PitNextLap*

Variable	r	p-valor	Sig.
Stint	+0.0870	0.0000	Sí
TyreLife	+0.2662	0.0000	Sí
Position	-0.0322	0.0000	Sí
LapTime (s)	-0.0212	0.0000	Sí
LapTime_Delta	+0.0036	0.2552	No
Cumulative_Degradation	-0.1223	0.0000	Sí
RaceProgress	+0.0992	0.0000	Sí
Normalized_TyreLife	+0.1254	0.0000	Sí
Position_Change	+0.0575	0.0000	Sí

H. Balanceo

El balanceo se aplicó únicamente sobre el 70% de los datos correspondiente al conjunto de entrenamiento. Esta decisión evita contaminar el conjunto de prueba con datos sintéticos y permite que la evaluación final refleje un escenario real.

Para tratar el desbalanceo se utilizó SMOTE, técnica que genera ejemplos sintéticos de la clase minoritaria a partir de sus

vecinos más cercanos. Esta estrategia permitió que los modelos aprendieran mejor los patrones asociados a la clase positiva, es decir, los casos donde sí ocurre un pit stop en la siguiente vuelta.

I. Transformaciones

Para los modelos basados en árboles, como Árbol de Decisión, Random Forest, Gradient Boosting, Bagging y AdaBoost, no se aplicó normalización, debido a que estos algoritmos no dependen de la escala de las variables.

Para modelos como KNN, SVM y Red Neuronal sí se aplicó normalización o estandarización, debido a que son sensibles a la escala de las variables. Además, la variable categórica *Compound* fue codificada para poder ser usada por los algoritmos de aprendizaje automático.

6. MODELAMIENTO, EVALUACIÓN E INTERPRETACIÓN

A. Configuración de métodos de Machine Learning

Se implementaron modelos supervisados y modelos de ensamble. Los modelos supervisados permiten aprender la relación entre las variables predictoras y la variable objetivo. Los modelos de ensamble combinan múltiples modelos base para mejorar estabilidad y rendimiento.

Table 7. Modelos implementados

Modelo	Descripción
Árbol de Decisión	Modelo interpretable basado en reglas de decisión.
KNN	Clasificador basado en vecinos cercanos; requiere normalización.
Red Neuronal	Modelo MLP capaz de capturar relaciones no lineales.
SVM	Clasificador basado en márgenes de separación.
SGD-SVM Rápido	Versión aproximada y más eficiente para datos grandes.
Random Forest	Ensamble de árboles de decisión mediante bagging.
Gradient Boosting	Ensamble secuencial que corrige errores de modelos anteriores.
Bagging	Ensamble que reduce varianza combinando modelos base.
AdaBoost	Ensamble adaptativo que asigna mayor peso a errores previos.

B. Validación cruzada

La validación cruzada se realizó sobre el conjunto de entrenamiento, correspondiente al 70% de los datos. Este procedimiento permitió evaluar la estabilidad de los modelos y reducir el riesgo de seleccionar un modelo que solo funcionara bien en una partición específica.

Debido al desbalanceo de clases, se utilizó una partición estratificada para conservar la proporción de la variable objetivo en cada fold. La métrica principal para comparar modelos fue ROC-AUC, complementada con accuracy, precision, recall y F1-score.

C. Análisis de medidas de calidad

Table 8. Medidas de calidad de los modelos

Modelo	Acc.	Prec.	Recall	F1	AUC
Árbol	0.7994	0.5772	0.7951	0.6688	0.8765
KNN	0.8874	0.7277	0.8919	0.8015	0.9547
NN	0.8314	0.6184	0.8832	0.7275	0.9250
SVM	0.6373	0.2623	0.2336	0.2471	0.5485
SGD-SVM	0.7290	0.4736	0.5700	0.5173	0.7506
Random Forest	0.8765	0.7113	0.8675	0.7817	0.9499
Gradient Boosting	0.8459	0.6743	0.7645	0.7166	0.9134
Bagging	0.8035	0.5831	0.8027	0.6755	0.8805
AdaBoost	0.7952	0.5713	0.7859	0.6616	0.8690

El modelo KNN obtuvo el mejor desempeño inicial, con ROC-AUC de 0.9547, F1-score de 0.8015 y recall de 0.8919. Esto indica

que fue muy efectivo para detectar casos donde sí ocurre un pit stop.

Random Forest también presentó un desempeño competitivo, con ROC-AUC de 0.9499, F1-score de 0.7817 y recall de 0.8675. Además, este modelo tiene ventajas para despliegue, ya que no requiere normalización y permite analizar importancia de variables.

La Red Neuronal obtuvo ROC-AUC de 0.9250, F1-score de 0.7275 y recall de 0.8832. Aunque detectó muchos casos positivos, su precisión fue menor que la de KNN y Random Forest.

El modelo SVM tradicional presentó el peor desempeño, con ROC-AUC de 0.5485 y F1-score de 0.2471, lo cual indica baja capacidad para separar correctamente las clases.

D. Selección del mejor modelo mediante ANOVA y Tukey

Para seleccionar los mejores modelos se aplicó ANOVA sobre los resultados de validación cruzada. Esta prueba permite determinar si existen diferencias estadísticamente significativas entre los modelos. Posteriormente, se aplicó la prueba post-hoc de Tukey HSD para comparar los modelos por pares.

Los resultados mostraron que KNN y Random Forest no presentaron diferencia estadísticamente significativa entre sí, con $p = 0.9995$. También se observó que Gradient Boosting y Random Forest no tuvieron diferencia significativa, con $p = 0.5752$. Sin embargo, al considerar simultáneamente ROC-AUC, F1-score y recall, se seleccionaron como tres mejores modelos KNN, Random Forest y Red Neuronal.

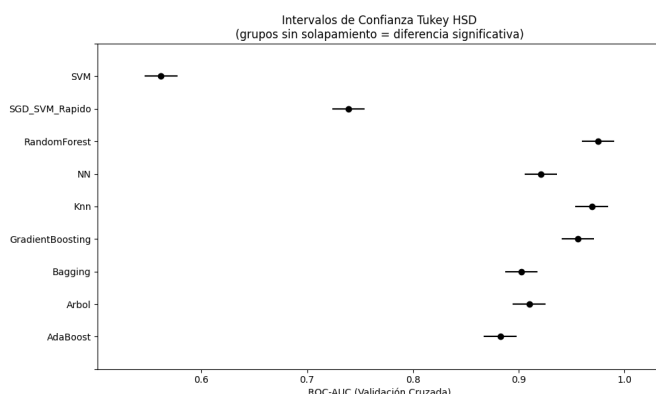


Fig. 3. Resultados de la prueba post-hoc de Tukey para comparación estadística de modelos.

Table 9. Tres mejores modelos antes de hiperparametrización

Modelo	ROC-AUC	F1	Recall
KNN	0.9547	0.8015	0.8919
Random Forest	0.9499	0.7817	0.8675
Red Neuronal	0.9250	0.7275	0.8832

E. Optimización de hiperparámetros con GridSearchCV

Los tres modelos seleccionados fueron optimizados mediante GridSearchCV. Este proceso evaluó distintas combinaciones de hiperparámetros y seleccionó la configuración con mejor desempeño según ROC-AUC.

Table 10. Mejores hiperparámetros con GridSearchCV

Modelo	Mejores hiperparámetros	CV AUC	Test AUC
KNN	metric=manhattan, n_neighbors=7, weights=distance	0.9805	0.9702
RF	max_depth=None, min_samples_split=2, n_estimators=200	0.9917	0.9778
NN	activation=relu, hidden_layer_sizes=(128,64), learning_rate_init=0.001	0.9454	0.9429

Table 11. Medidas de calidad después de GridSearchCV

Métrica	KNN	RF	NN
Accuracy	0.9215	0.9369	0.8777
Precision	0.8100	0.8745	0.7248
Recall	0.9040	0.8783	0.8382
F1-score	0.8544	0.8764	0.7774
ROC-AUC	0.9702	0.9778	0.9429

Después de GridSearchCV, el mejor modelo fue Random Forest, con accuracy de 0.9369, precision de 0.8745, recall de 0.8783, F1-score de 0.8764 y ROC-AUC de 0.9778. Aunque KNN obtuvo un recall ligeramente superior, Random Forest presentó mejor equilibrio global y mayor ROC-AUC.

F. Optimización bayesiana

Luego se aplicó optimización bayesiana al mejor modelo obtenido con GridSearchCV, es decir, Random Forest. Esta técnica busca hiperparámetros de forma más eficiente utilizando los resultados de iteraciones anteriores.

La optimización bayesiana encontró como mejores parámetros: max_depth=30, min_samples_leaf=1, min_samples_split=2 y n_estimators=300. Con esta configuración se obtuvo un ROC-AUC de validación cruzada de 0.9917, un ROC-AUC en prueba de 0.9776, F1-score de 0.8732 y recall de 0.8752.

Table 12. Comparación entre GridSearchCV y optimización bayesiana

Modelo	Test ROC-AUC
Random Forest con GridSearchCV	0.9778
Random Forest con optimización bayesiana	0.9776

Aunque la diferencia fue mínima, se seleccionó el modelo Random Forest optimizado con GridSearchCV, ya que obtuvo el mayor ROC-AUC en el conjunto de prueba.

```

Modelo guardado en: modelo_f1.pkl
Tipo de modelo: RandomForestClassifier
Requiere normalización: False

```

	Arbol	Knn	NN	SVM	SGD_SVM_Rapido	RandomForest
accuracy	0.7994	0.8874	0.8314	0.6373	0.7290	0.8765
precision	0.5772	0.7277	0.6184	0.2623	0.4736	0.7113
recall	0.7951	0.8919	0.8832	0.2336	0.5700	0.8675
f1	0.6688	0.8015	0.7275	0.2471	0.5173	0.7817
roc_auc	0.8765	0.9547	0.9250	0.5485	0.7506	0.9499

	GradientBoosting	Bagging	AdaBoost
accuracy	0.8459	0.8035	0.7952
precision	0.6743	0.5831	0.5713
recall	0.7645	0.8027	0.7859
f1	0.7166	0.6755	0.6616
roc_auc	0.9134	0.8805	0.8690

Fig. 4. Comparación de desempeño de los modelos evaluados.

7. PIPELINE FINAL Y DESPLIEGUE

A. Construcción del pipeline de preparación

Después de seleccionar el modelo final, se construyó una estructura de preparación de datos orientada al despliegue. El objetivo de esta etapa fue garantizar que los datos ingresados por el usuario recibieran el mismo tratamiento aplicado durante el entrenamiento antes de ser enviados al modelo predictivo.

Para esto, se definieron transformadores personalizados compatibles con `sklearn`. El primer transformador fue `LapTimeCleaner`, encargado de corregir valores atípicos en la variable `LapTime` (s) cuando el tiempo de vuelta superaba los 150 segundos. En el entrenamiento, este proceso utilizó la mediana por carrera; en escenarios de inferencia, cuando la variable `Race` no está disponible, se utiliza una mediana global aprendida durante el ajuste.

El segundo transformador fue `FeatureEngineer`, encargado de realizar la ingeniería de variables. Este paso codifica la variable categórica `Compound` mediante una representación ordinal, calcula `Normalized_TyreLife` a partir de `TyreLife` y genera variables derivadas como `LapTime_Delta` y `Position_Change` cuando estas no se encuentran disponibles en los datos nuevos.

Finalmente, se incorporó un selector de columnas, denominado `ColumnSelector`, que organiza las variables en el orden exacto esperado por el modelo. Este paso es importante porque garantiza que, durante el despliegue, el modelo reciba las mismas variables y en la misma estructura utilizada durante el entrenamiento.

Table 13. Componentes de preparación para el despliegue

Componente	Función
LapTimeCleaner	Corrige valores atípicos en LapTime (s) mayores a 150 segundos.
FeatureEngineer	Codifica Compound, calcula Normalized_TyreLife y genera variables derivadas si no existen.
ColumnSelector	Selecciona y ordena las variables requeridas por el modelo.
MinMaxScaler	Escala las variables numéricas para mantener la misma transformación usada en entrenamiento.
Modelo final	Aplica el clasificador seleccionado para predecir si habrá pit stop en la siguiente vuelta.

B. Entrenamiento final y almacenamiento de artefactos

El conjunto de datos fue dividido inicialmente en 70% para entrenamiento y 30% para prueba, manteniendo la proporción de clases mediante una partición estratificada. El preprocesamiento

se ajustó únicamente sobre el conjunto de entrenamiento para evitar fuga de información. Posteriormente, se aplicó SMOTE sobre el 70% de entrenamiento ya procesado, debido a que el balanceo debe realizarse solo sobre los datos de entrenamiento y no sobre el conjunto de prueba.

Después de evaluar el desempeño del modelo seleccionado en el 30% de prueba, se realizó un reentrenamiento final utilizando el 100% de los datos disponibles. Para este entrenamiento final, primero se aplicó el preprocesamiento completo, luego se balancearon los datos con SMOTE y finalmente se entrenó el modelo seleccionado.

Para revisar si el modelo final presentaba sobreajuste, se compararon las métricas obtenidas en el conjunto de entrenamiento y en el conjunto de prueba. En ROC-AUC, el modelo obtuvo un valor de 0.9965 en entrenamiento y 0.9999 en prueba.

Estos resultados muestran que el modelo mantiene un desempeño muy alto en ambos conjuntos. La diferencia entre el ROC-AUC de entrenamiento y prueba es muy pequeña, aproximadamente de 0.0034, por lo que no se evidencia un sobreajuste fuerte según esta métrica. Además, el ROC-AUC cercano a 1 indica que el modelo tiene una capacidad muy alta para diferenciar entre las clases “Pit” y “No Pit”.

Sin embargo, debido a que las métricas del conjunto de prueba son incluso superiores a las de entrenamiento, es importante interpretar el resultado con precaución.

El modelo final se almacenó en un archivo `pipeline_f1.pkl`. En este archivo se guardaron los artefactos necesarios para el despliegue, principalmente el escalador utilizado durante el preprocesamiento y el modelo entrenado. Esta decisión permite que la aplicación de Streamlit cargue directamente los componentes ya entrenados, sin necesidad de repetir todo el proceso de modelamiento.

Table 14. Artefactos guardados para el despliegue

Artefacto	Descripción
scaler	Transformador utilizado para escalar las variables de entrada antes de la predicción.
modelo	Clasificador final entrenado para predecir la variable <code>PitNextLap</code> .
pipeline_f1.pkl	Archivo serializado que contiene los artefactos usados por la aplicación de despliegue.

C. Despliegue con Streamlit

El despliegue se realizó mediante una interfaz gráfica desarrollada en Streamlit. Esta aplicación permite que el usuario ingrese manualmente los datos de una vuelta actual y obtenga una predicción sobre si el piloto debería realizar un pit stop en la siguiente vuelta.

La aplicación carga los artefactos del modelo desde Hugging Face usando la función `hf_hub_download`. El archivo utilizado es `pipeline_f1.pkl`, alojado en el repositorio `CarlosC4/modelo-despliegue-f1`. A partir de este archivo se recuperan el escalador y el modelo final entrenado.

En la interfaz se solicitan variables como el compuesto de neumático, el número de stint, la vida del neumático, la posición en carrera, el tiempo de vuelta, el delta del tiempo de vuelta, la degradación acumulada, el progreso de carrera y el cambio de posición. Con estos valores, la aplicación construye un arreglo de entrada en el mismo orden usado durante el entrenamiento.

Antes de predecir, la aplicación realiza las transformaciones necesarias: codifica el compuesto de neumático, calcula la vida normalizada del neumático, organiza las variables en el orden correcto, aplica el escalamiento con el scaler cargado y finalmente envía los datos al modelo.



Fig. 5. Interfaz en la aplicación Streamlit.

Table 15. Variables de entrada en la aplicación Streamlit

Variable	Descripción
Compound	Compuesto de neumático seleccionado por el usuario.
Stint	Número de stint en el que se encuentra el piloto.
TyreLife	Cantidad de vueltas recorridas con el neumático actual.
Position	Posición actual del piloto en la carrera.
LapTime (s)	Tiempo de vuelta actual medido en segundos.
Normalized_TyreLife	Variable calculada automáticamente a partir de TyreLife.
Position_Change	Cambio de posición del piloto respecto a la vuelta anterior.

D. Funcionamiento de la predicción

Cuando el usuario presiona el botón Predecir Pit Stop, la aplicación realiza el proceso de inferencia. Primero, transforma los datos ingresados en un arreglo numérico compatible con el modelo. Luego aplica el escalamiento correspondiente y calcula dos salidas: la clase predicha y la probabilidad estimada de pit stop.

La clase 0 indica que el modelo recomienda continuar en pista, mientras que la clase 1 indica que el modelo recomienda realizar un pit stop en la siguiente vuelta. Además, la aplicación muestra la probabilidad asociada a la clase positiva, lo que permite interpretar el nivel de confianza del modelo.

Table 16. Interpretación de la salida del modelo

Predicción	Resultado	Interpretación
0	Continuar en pista	El modelo estima que el piloto no realizará pit stop en la siguiente vuelta.
1	Pit stop recomendado	El modelo estima que el piloto tiene alta probabilidad de realizar pit stop en la siguiente vuelta.

E. Importancia del despliegue

El despliegue permite convertir el modelo predictivo en una herramienta funcional. En lugar de limitarse a los resultados del notebook, la aplicación permite que una persona ingrese nuevos escenarios de carrera y obtenga una predicción inmediata. Esto facilita la interpretación del modelo y demuestra cómo el proceso de minería de datos puede llevarse a un entorno de uso práctico.

Además, al cargar el modelo desde Hugging Face, se facilita la portabilidad del proyecto, ya que el archivo entrenado no depende del entorno local de desarrollo. De esta forma, el modelo puede reutilizarse desde diferentes computadores o entornos, siempre que se tenga acceso al repositorio donde está almacenado.

F. Monitoreo y mantenimiento

Para mantener el modelo en producción, se recomienda comparar periódicamente las predicciones realizadas por la aplicación con resultados reales de carrera. Esto permitiría identificar si el modelo conserva su desempeño o si empieza a perder precisión debido a cambios en las estrategias, reglamentos, neumáticos o comportamiento de los equipos.

También se recomienda actualizar el modelo con nuevas temporadas de datos, revisar la calidad de las variables de entrada y validar que las transformaciones aplicadas en Streamlit coincidan con las usadas durante el entrenamiento. Este monitoreo es importante para evitar que el modelo quede desactualizado frente a nuevas condiciones de competencia.

Table 17. Cronograma de mantenimiento y reentrenamiento

Actividad	Frecuencia o condición
Reentrenamiento del modelo	Al inicio de cada temporada o cuando disminuya el desempeño del modelo.
Actualización de datos	Cuando se incorporen nuevas carreras, pilotos, circuitos o temporadas.
Revisión de variables	Cuando cambien las reglas, tipos de neumáticos o estructura del dataset.
Validación del despliegue	Cada vez que se actualice el archivo <code>pipeline_f1.pkl</code> o la aplicación Streamlit.
Comparación con resultados reales	Después de nuevas carreras, para verificar si las predicciones coinciden con los pit stops observados.

G. Pipeline final del modelo

Para garantizar que el modelo pudiera ser utilizado correctamente en el despliegue, se construyó un pipeline final que integra las operaciones de preparación de los datos y el modelo predictivo seleccionado. Este pipeline permite que los datos ingresados por el usuario en la aplicación Streamlit pasen por las mismas transformaciones aplicadas durante el entrenamiento, evitando inconsistencias entre la etapa de modelamiento y la etapa de despliegue.

El pipeline final quedó conformado por un preprocesador, encargado de corregir valores atípicos, generar variables derivadas, seleccionar columnas y escalar variables numéricas, seguido del modelo final `RandomForestClassifier`, seleccionado después del proceso de evaluación, comparación estadística e hiperparametrización.

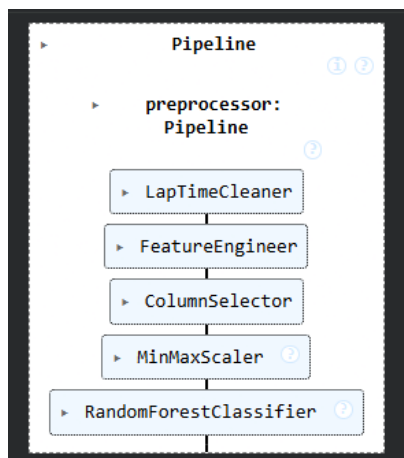


Fig. 6. Pipeline final integrado con preprocesamiento y modelo Random Forest.

Como se observa en la Figura 6, el modelo final no se almacena de manera aislada, sino junto con las transformaciones necesarias para recibir datos nuevos. Esto facilita el despliegue y reduce el riesgo de errores al momento de realizar predicciones en la interfaz gráfica.

8. CONCLUSIONES

El objetivo general del proyecto se cumplió, ya que se desarrolló un modelo predictivo de minería de datos en Python capaz de estimar si un piloto de Fórmula 1 realizará un pit stop en la siguiente vuelta. Además, el modelo final fue llevado a una interfaz gráfica en Streamlit, lo que permitió convertir el análisis realizado en una herramienta funcional de predicción.

La preparación de datos fue una etapa fundamental para mejorar la calidad del conjunto utilizado. El tratamiento de valores nulos en la variable Compound, la corrección de valores atípicos en LapTime (s) y la transformación de variables permitieron construir una base más consistente para el entrenamiento de los modelos. Esto demuestra que la calidad del preprocesamiento influye directamente en la confiabilidad de los resultados obtenidos.

El análisis de la variable objetivo permitió identificar un desbalance entre las clases, debido a que la mayoría de las vueltas no están asociadas a un pit stop en la siguiente vuelta. Por esta razón, el balanceo aplicado únicamente sobre el conjunto de entrenamiento fue una decisión adecuada, ya que permitió mejorar el aprendizaje de los modelos sin contaminar el conjunto de prueba.

La comparación entre modelos supervisados y modelos de ensamble permitió evidenciar que no todos los algoritmos responden de la misma manera ante este problema. Modelos como KNN, Random Forest y la Red Neuronal mostraron los mejores resultados iniciales, mientras que otros modelos, como SVM, tuvieron un desempeño inferior para este conjunto de datos.

El uso de diferentes métricas de evaluación permitió realizar una selección más completa del modelo. En este problema, el accuracy por sí solo no era suficiente, debido al desbalance de clases. Por eso, métricas como *precision*, *recall*, F1-score y ROC-AUC fueron necesarias para interpretar mejor la capacidad del modelo para detectar correctamente los casos en los que sí se aproxima un pit stop.

La aplicación de ANOVA y la prueba post-hoc de Tukey permitió comparar estadísticamente los modelos evaluados y

seleccionar los tres mejores con mayor fundamento. Esta etapa fue importante porque la elección del modelo no se basó únicamente en observar las métricas, sino también en validar si existían diferencias estadísticamente significativas entre los desempeños obtenidos.

Después del proceso de hiperparametrización, Random Forest optimizado con GridSearchCV fue seleccionado como el mejor modelo final. Este modelo alcanzó un ROC-AUC de 0.9778 y un F1-score de 0.8764, lo que indica una alta capacidad para diferenciar entre vueltas donde habrá pit stop y vueltas donde el piloto continuará en pista. Aunque la optimización bayesiana produjo resultados muy similares, GridSearchCV presentó un rendimiento ligeramente superior.

La construcción del pipeline final permitió integrar las operaciones de preparación y el modelo seleccionado, asegurando que los datos nuevos recibieran un tratamiento coherente con el utilizado durante el entrenamiento. Esto es clave para que el modelo pueda ser usado fuera del notebook sin perder consistencia en las predicciones.

El despliegue en Streamlit permitió cumplir la última fase del proyecto, al presentar el modelo en una interfaz gráfica sencilla e interactiva. Esta aplicación facilita que un usuario ingrese las condiciones de una vuelta actual y obtenga una recomendación inmediata sobre la posibilidad de realizar un pit stop en la siguiente vuelta.

9. TRABAJO FUTURO

Como trabajo futuro se recomienda ampliar el conjunto de datos incluyendo más temporadas, condiciones climáticas, presencia de safety car, información del circuito, telemetría avanzada y estrategias específicas por equipo. También sería útil comparar el modelo con técnicas de series de tiempo o modelos secuenciales, ya que las decisiones de pit stop dependen del comportamiento acumulado durante varias vueltas.

Además, la aplicación de Streamlit podría mejorarse permitiendo cargar archivos CSV con múltiples vueltas, visualizar probabilidades por piloto, comparar estrategias y generar reportes automáticos de predicción.

REFERENCES

1. FastF1. *FastF1 Python package documentation*. Disponible en: <https://docs.fastf1.dev/>
2. Scikit-learn. *Machine Learning in Python*. Disponible en: <https://scikit-learn.org/>
3. Streamlit. *Streamlit documentation*. Disponible en: <https://docs.streamlit.io/>
4. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. y Wirth, R. *CRISP-DM 1.0: Step-by-step data mining guide*. 2000.
5. The pandas development team. *pandas documentation*. Disponible en: <https://pandas.pydata.org/docs/>